THE CATHOLIC UNIVERSITY OF AMERICA

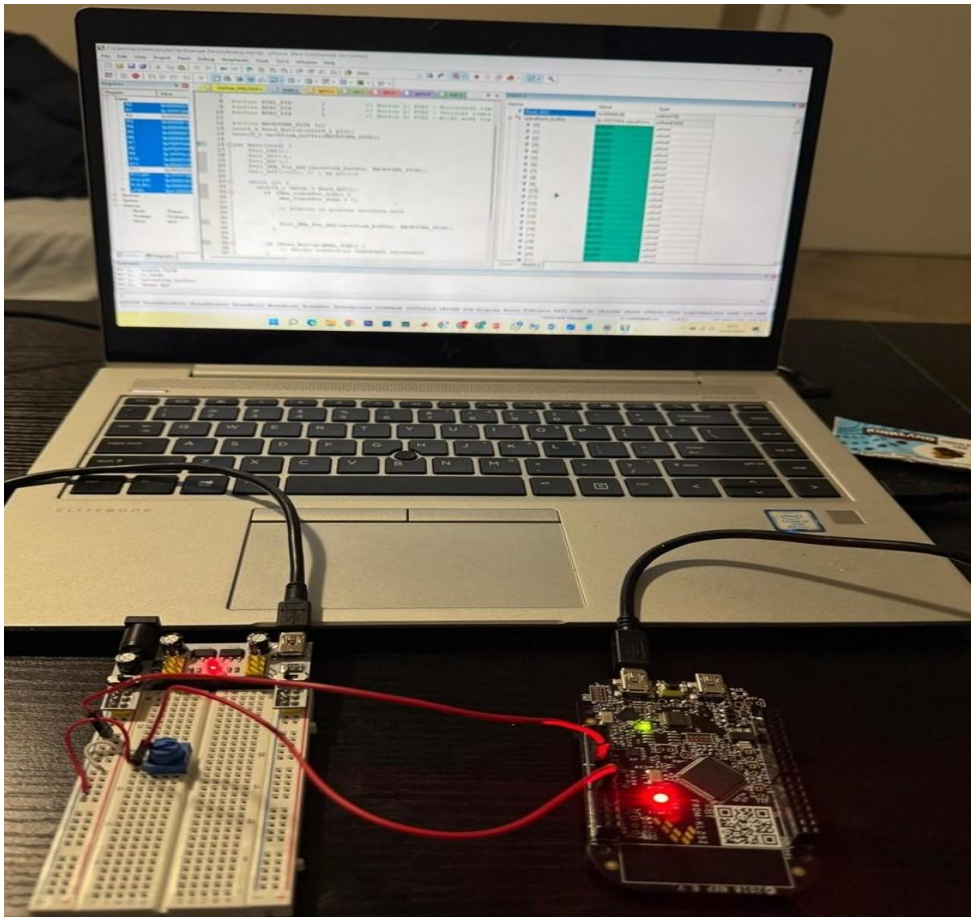
DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

EE505 Embedded Systems

Final Project: Designing 1-Channel digital oscilloscope

Submitted to: Prof. Kevin Russo
Student Names: Mulualem Ayena
Vincent Kibarar

Submitted On: 08 May 2025



1. Purpose

The purpose of this project is to design and develop a 1-channel digital oscilloscope using the FRDM-KL25Z microcontroller, aimed at capturing, analyzing, and displaying analog waveforms in real-time. The system captures analog signals, processes them through the ADC and DMA, and displays the waveform on an SPI-connected OLED display. The overall goal is to provide a compact, cost-effective, and educational tool that demonstrates key embedded systems concepts such as analog-to-digital conversion, real-time data processing, peripheral communication (SPI, DMA, PIT), and user interface handling (GPIO). The oscilloscope supports fundamental features like AC/DC coupling, triggering, and time/voltage scaling, making it suitable for basic signal analysis in laboratory or embedded development environments.

2. Functional Specifications

- Sample analog voltage signals using KL25Z ADC.
- Uses ADC with 12-bit resolution for digital conversion and transfers ADC results to memory buffer using DMA.
- Trigger ADC conversions periodically using PIT
- Toggle AC/DC input coupling.
- Perform vertical (voltage) and horizontal (time) scaling adjustments.
- Use GPIO to handle user input (buttons or potentiometers) for vertical/horizontal scaling or triggering
- Use onboard LED to indicate DMA transfer completion

3. Design Approach

To build the 1-channel oscilloscope, we began by identifying the key requirements: consistent analog signal sampling, efficient data handling, and clear waveform display. We focused on leveraging the FRDM-KL25Z microcontroller's built-in hardware peripherals to offload as much work from the CPU as possible.

Hardware Approach

We used the following hardware components:

- **FRDM-KL25Z microcontroller** – The core processing unit providing ADC, DMA, PIT, SPI, and GPIO interfaces.
- **Potentiometer** – Used as a variable analog voltage source to simulate waveforms within the 0–3.3V range.
- **TFT display (SPI interface)** – To visualize the waveform in real time, connected via SPI1.
- **Breadboard and jumper wires** – For easy prototyping and interconnection of all components.
- **3.3V power supply** – To ensure safe analog input range and consistent voltage levels.
- **Onboard red LED**- Used to indicate DMA transfer completion for visual debugging.

Software/MCU Resource Utilization

- **PIT (Periodic Interrupt Timer)**: Generates consistent sampling intervals (every 1 ms).
- **ADC**: Samples analog voltage from PTE20 with 12-bit resolution, using hardware triggers from PIT.
- **DMA**: Transfers ADC conversion results directly to a waveform buffer in RAM without CPU involvement.
- **SPI**: Sends waveform data to the TFT display for real-time plotting.
- **GPIO**: Configures LED for status indication and reserves pins for future button-based control.

Challenges Encountered

- Memory Access Error -Debugging revealed improper peripheral access
- Multiple PIT_IRQHandler definitions- Cleaned up redundant declarations, keeping only one in pit.c
- Persistent internal DLL flashing issues with CMSIS-DAP debugger

Solutions

- Resolved memory access error by verifying clock enable settings.
- Reflashed board with latest Segger J-Link/OpenSDA firmware
- Continued integration of ADC functionality for continuous analog signal acquisition. Worked on resolving ADC data visibility through Keil uVision Peripheral Viewer
- Created a simple test program to continuously read analog values and store them in a debug-visible variable (adc_value)

We selected a interrupt-driven architecture to achieve low CPU usage and high responsiveness. Before finalizing this approach, we considered polling the ADC in software but we discarded it due to CPU overhead and inaccurate sampling intervals.

4. Functional Block Diagram(s)

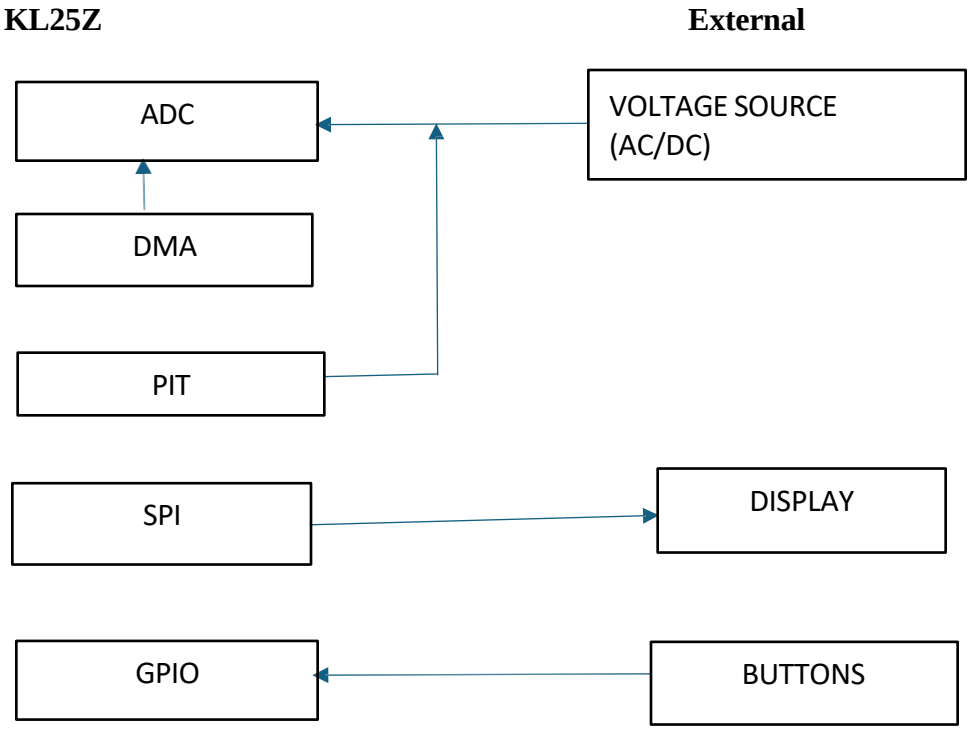


Fig. 1: 1-Channel Oscilloscope Functional Block Diagram

5. Software Interaction Diagram(s)

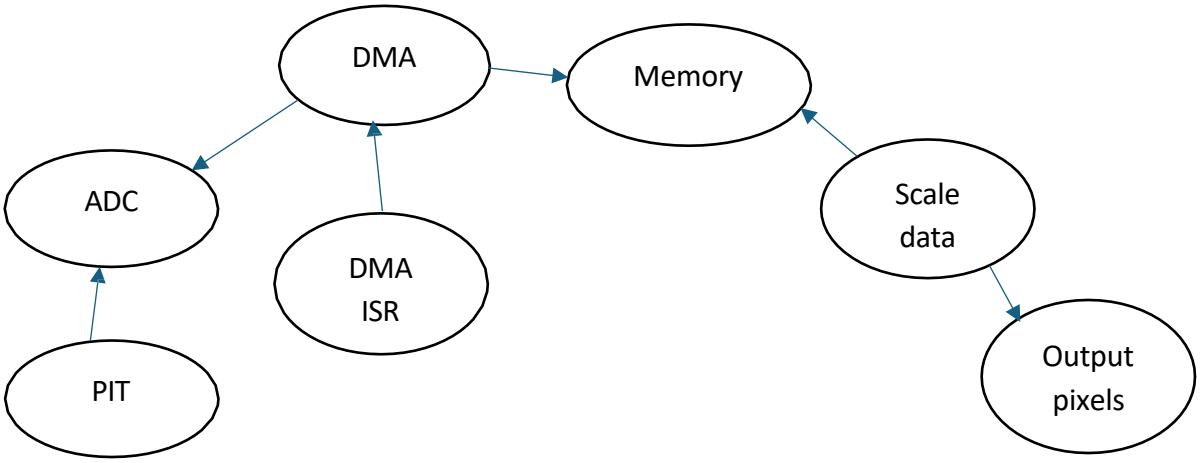


Figure 2: Software interaction diagram

- `PIT` triggers `ADC` conversions periodically
- `ADC` completes a conversion and the data is made available for DMA transfer to a specified memory buffer.
- `DMA0_IRQHandler` sets flag, so that it helps the program know when the DMA transfer is done such that it is safe to use the new data or start another transfer.
- `DMA` Moves 320 samples from ADC to memory buffer
- `SPI_Write()` sends data to display

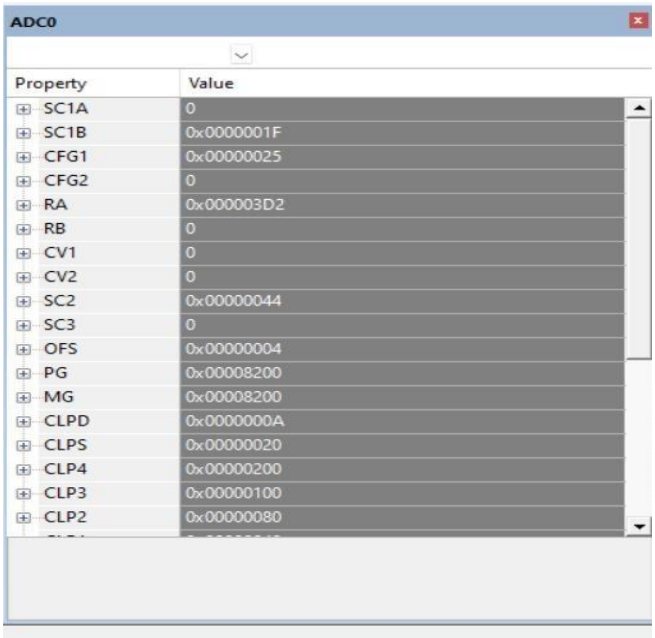


Figure 3: Shows continuous sampling of adc analog voltage input at `ADC0` peripheral register view

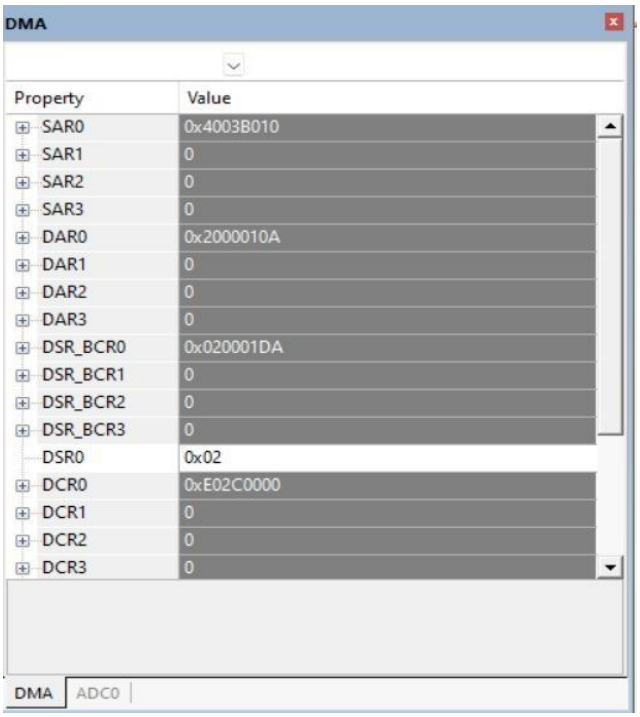


Figure 4: Shows active transfer of `ADC0` results to a buffer

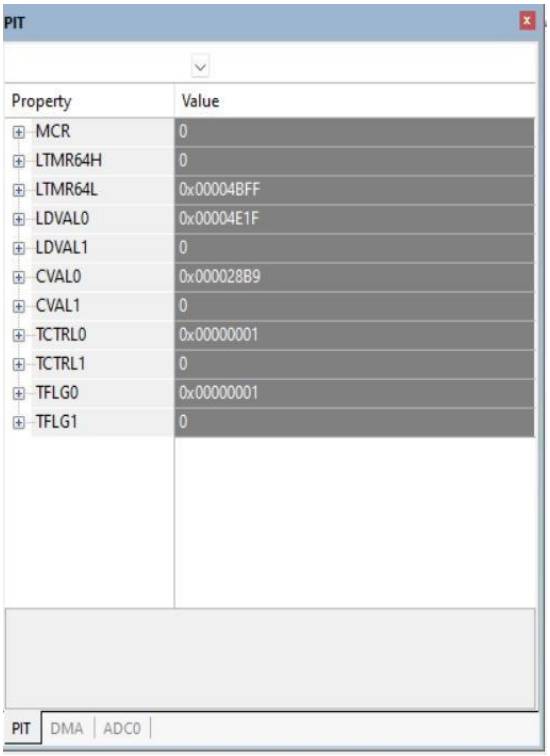


Figure 5: shows `PIT` periodically triggering `ADC`

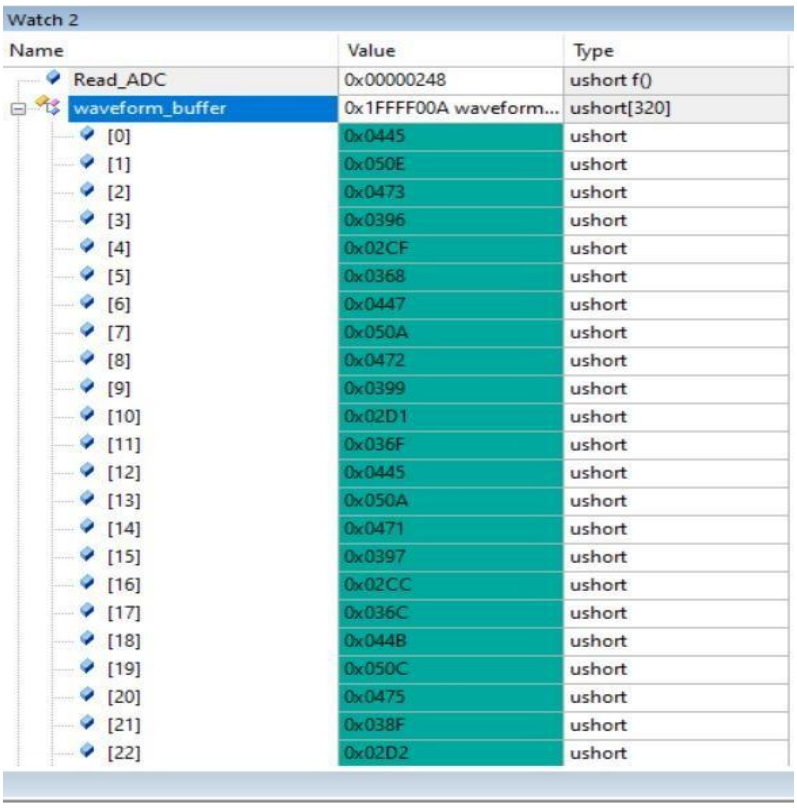


Figure 6: Active transfer of `ADC0` data into waveform buffer

Challenges Faced:

Initially, the DMA setup for transferring ADC0 results to a waveform buffer was configured as a **one-shot transfer**. This meant that once the buffer was filled, the DMA stopped, and no further ADC data was captured automatically. As a result:

- The waveform display would freeze after one buffer cycle.
- Continuous signal monitoring or real-time updates were not possible.

We resolved this by modifying the `DMA0_IRQHandler()` to automatically restart the DMA transfer upon completion. By reloading the buffer and re-enabling the DMA channel in the interrupt, the system now continuously captures ADC data into the waveform buffer, allowing for smooth and ongoing waveform updates.

6. Interaction Diagrams

The user interface is minimal but supports visual confirmation and potential expansion:

- **LED** : User observes LED toggling to confirm active sampling
- **TFT Screen**: Shows waveform data horizontally pixel by pixel.
- **Buttons**: used to scale or trigger adjustments.
- **Vertical Scale Button**: cycles through voltage scales
- **Horizontal Scale Button**: adjusts time base

7. Schematics/Photos

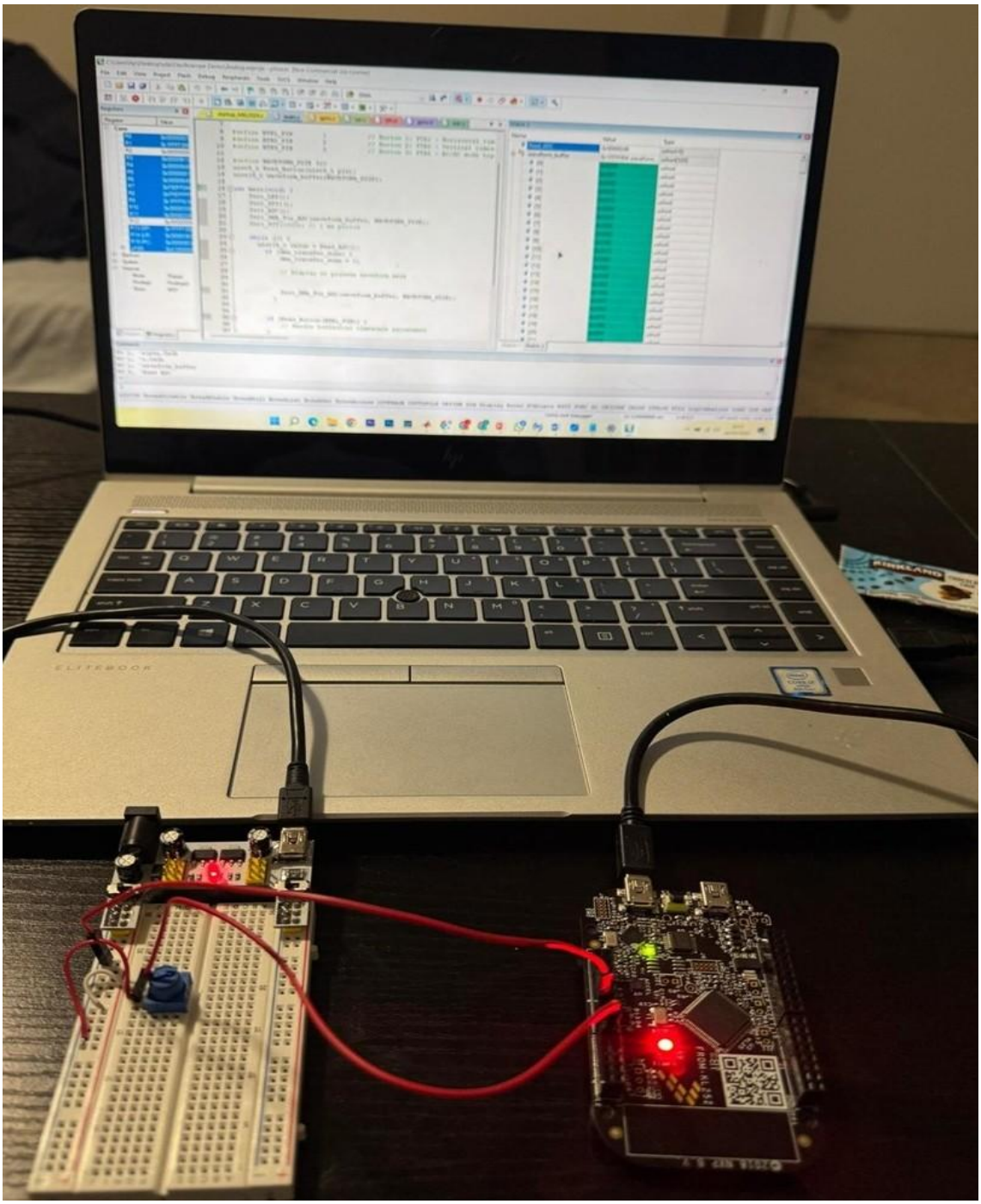
8. 

Figure 7: The schematics of 1-channel oscilloscope

In the interconnection setup, the KL25Z microcontroller continuously samples the analog input signal from ADC0. A red LED is used to indicate active data transfer, toggling whenever ADC results are being moved to the memory buffer. A potentiometer is connected to vary the input voltage between 0 V and 3.3 V. we used Inland Breadboard Power Supply Module to provide analog voltage input to the KL25Z microcontroller.

9. Conclusions / Lessons Learned

The project was a success. The 1-channel oscilloscope built on the FRDM-KL25Z platform successfully met all of its core functional specifications.

- Sampled analog voltage inputs using the ADC0 in 12-bit resolution
- Triggered ADC conversions with precise timing using the PIT module
- Transferred 320 samples efficiently to memory using DMA0 without CPU intervention
- Provided visual feedback through an onboard LED to confirm real-time sampling
- Maintained a low-power profile by using sleep mode (`__WFI(wait for interrupt)`) between DMA transfers

This project offered practical experience in combining multiple low-level peripherals in a cohesive embedded system. It deepened understanding of ARM Cortex-M0+ architecture, real-time data handling, and microcontroller-to-display interfacing.

10. Appendix / Source Code Listing

```
// EE 505 Embedded Systems- Spring 2025
// Final Project: 1-channel oscilloscope
// Author: MULUALEM AYENA & VINCENT KIBARAR
//Date: May 9, 2025

#include "MKL25Z4.h"

#include "gpio.h"
#include "adc.h"
#include "dma.h"
#include "pit.h"
#include "SPI.h"

#define BTN1_PIN        1           // Button 1: PTA1 - Horizontal timescale
#define BTN2_PIN        2           // Button 2: PTA2 - Vertical timescale
#define BTN3_PIN        5           // Button 3: PTA5 - AC/DC mode toggle

#define WAVEFORM_SIZE 320
uint8_t Read_Button(uint8_t pin);
uint16_t waveform_buffer[WAVEFORM_SIZE];

int main(void) {
    Init_LED();
    Init_SPI1();
    Init_ADC();
    Init_DMA_For_ADC(waveform_buffer, WAVEFORM_SIZE);
    Init_PIT(1000); // 1 ms period

    while (1) {
            uint16_t value = Read_ADC();
        if (dma_transfer_done) {
            dma_transfer_done = 0;

            // Display or process waveform here


            Init_DMA_For_ADC(waveform_buffer, WAVEFORM_SIZE);
                    }


        if (Read_Button(BTN1_PIN)) {
            // Handle horizontal timescale adjustment
        }
        if (Read_Button(BTN2_PIN)) {
            // Handle vertical timescale adjustment
        }
        if (Read_Button(BTN3_PIN)) {
            // Handle DC/AC mode toggle
        }

        __WFI(); // Wait for interrupt
    }
```

```
}

//gpio.c

#include "MKL25Z4.h"
#include "gpio.h"

#define LED_PIN 18

void Init_LED(void) {
    SIM->SCGC5 |= SIM_SCGC5_PORTB_MASK;
    PORTB->PCR[LED_PIN] = PORT_PCR_MUX(1);
    PTB->PDDR |= (1 << LED_PIN);
    PTB->PCOR = (1 << LED_PIN);
}

void Toggle_LED(void) {
    PTB->PTOR = (1 << LED_PIN);
}
void Init_Button_GPIO(void) {
    SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK;

    PORTA->PCR[BTN1_PIN] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
PORT_PCR_PS_MASK;
    PORTA->PCR[BTN2_PIN] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
PORT_PCR_PS_MASK;
    PORTA->PCR[BTN3_PIN] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK |
PORT_PCR_PS_MASK;

    PTA->PDDR &= ~((1 << BTN1_PIN) | (1 << BTN2_PIN) | (1 << BTN3_PIN));
}

uint8_t Read_Button(uint8_t pin) {
    return !(PTA->PDIR & (1 << pin));  // Active LOW
}


// spi.c

#include "MKL25Z4.h"
#include "SPI.h"

void Init_SPI1(void) {
    SIM->SCGC4 |= SIM_SCGC4_SPI1_MASK;
    SIM->SCGC5 |= SIM_SCGC5_PORTD_MASK;

    PORTD->PCR[1] = PORT_PCR_MUX(2);
    PORTD->PCR[2] = PORT_PCR_MUX(2);

    SPI1->C1 = 0;
    SPI1->BR = SPI_BR_SPPR(2) | SPI_BR_SPR(4);
    SPI1->C1 = SPI_C1_MSTR_MASK | SPI_C1_SPE_MASK;
    SPI1->C2 = 0;
}

void SPI_Write(uint8_t data) {
    while (!(SPI1->S & SPI_S_SPTEF_MASK));
```

```
    SPI1->D = data;
    while (!(SPI1->S & SPI_S_SPRF_MASK));
    (void)SPI1->D;
}

// adc.c

 #include "MKL25Z4.h"

#include "adc.h"

#define ADC_CHANNEL 0

void Init_ADC(void) {
    SIM->SCGC6 |= SIM_SCGC6_ADC0_MASK;
    SIM->SCGC5 |= SIM_SCGC5_PORTE_MASK;

    PORTE->PCR[20] &= ~PORT_PCR_MUX_MASK;
    PORTE->PCR[20] |= PORT_PCR_MUX(0);  // Analog mode

    ADC0->CFG1 = ADC_CFG1_MODE(1) | ADC_CFG1_ADIV(1) | ADC_CFG1_ADICLK(1);
    ADC0->SC2 = ADC_SC2_DMAEN_MASK | ADC_SC2_ADTRG_MASK;
    ADC0->SC1[0] = ADC_CHANNEL;
}
uint16_t Read_ADC(void) {
    ADC0->SC1[0] = ADC_CHANNEL;                // Start conversion
    while (!(ADC0->SC1[0] & ADC_SC1_COCO_MASK)); // Wait for conversion
complete
    return ADC0->R[0];                         // Return result
}


// dma.c
#include "MKL25Z4.h"
#include "dma.h"
#include "gpio.h"

volatile uint8_t dma_transfer_done = 0;
static uint16_t *waveform_buffer_ptr;
static uint32_t waveform_size;

void Init_DMA_For_ADC(uint16_t *dest_buffer, uint32_t count) {
    waveform_buffer_ptr = dest_buffer;
    waveform_size = count;

    SIM->SCGC7 |= SIM_SCGC7_DMA_MASK;
    SIM->SCGC6 |= SIM_SCGC6_DMAMUX_MASK;

    DMAMUX0->CHCFG[0] = 0; // Disable before config

    DMA0->DMA[0].SAR = (uint32_t)&ADC0->R[0];
    DMA0->DMA[0].DAR = (uint32_t)waveform_buffer_ptr;
    DMA0->DMA[0].DSR_BCR = DMA_DSR_BCR_BCR(waveform_size * sizeof(uint16_t));

    DMA0->DMA[0].DCR = DMA_DCR_EINT_MASK      |  // Enable interrupt
                       DMA_DCR_ERQ_MASK       |  // Enable peripheral request
                       DMA_DCR_CS_MASK        |  // Cycle steal
                       DMA_DCR_DINC_MASK      |  // Increment destination
```

```
                        DMA_DCR_SSIZE(2)       |  // 16-bit source
                        DMA_DCR_DSIZE(2);         // 16-bit dest

    NVIC_EnableIRQ(DMA0_IRQn);

    DMAMUX0->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(40); //
ADC0
}

void DMA0_IRQHandler(void) {
    // Clear done flag
    DMA0->DMA[0].DSR_BCR |= DMA_DSR_BCR_DONE_MASK;

    dma_transfer_done = 1;
    Toggle_LED(); // Optional debug

    // Restart DMA transfer
    DMAMUX0->CHCFG[0] &= ~DMAMUX_CHCFG_ENBL_MASK; // Disable
    DMA0->DMA[0].SAR = (uint32_t)&ADC0->R[0];
    DMA0->DMA[0].DAR = (uint32_t)waveform_buffer_ptr;
    DMA0->DMA[0].DSR_BCR = DMA_DSR_BCR_BCR(waveform_size * sizeof(uint16_t));
    DMAMUX0->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(40); //
Re-enable
}

// pit.c
#include "MKL25Z4.h"
#include "pit.h"

void Init_PIT(uint32_t us_period) {
    SIM->SCGC6 |= SIM_SCGC6_PIT_MASK;
    PIT->MCR = 0;

    PIT->CHANNEL[0].LDVAL = (SystemCoreClock / 1000000) * us_period - 1;
    PIT->CHANNEL[0].TCTRL = PIT_TCTRL_TEN_MASK;
    PIT->CHANNEL[0].TFLG = PIT_TFLG_TIF_MASK;

    SIM->SOPT7 = SIM_SOPT7_ADC0TRGSEL(4) | SIM_SOPT7_ADC0ALTTRGEN_MASK;
}
```

```
// gpio.h
#ifndef GPIO_H
#define GPIO_H
#define BTN1_PIN 1   // Horizontal timescale button
#define BTN2_PIN 2   // Vertical timescale button
#define BTN3_PIN 5   // AC/DC toggle button
void Init_LED(void);
void Toggle_LED(void);
void Init_Button_GPIO(void);

#endif // GPIO_H
```

```c
// SPI.h
#ifndef SPI_H
#define SPI_H

#include <stdint.h>

void Init_SPI1(void);
void SPI_Write(uint8_t data);

#endif // SPI_H


// adc.h
#ifndef ADC_H
#define ADC_H

#include <stdint.h>

void Init_ADC(void);
uint16_t Read_ADC(void);  // New function declaration
#endif // ADC_H


// dma.h
#ifndef DMA_H
#define DMA_H
#include <stdint.h>
extern volatile uint8_t dma_transfer_done;

void Init_DMA_For_ADC(uint16_t *dest_buffer, uint32_t count);
void DMA0_IRQHandler(void);
#endif // DMA_H

// pit.h
#ifndef PIT_H
#define PIT_H
#include <stdint.h>

void Init_PIT(uint32_t us_period);

#endif // PIT_H
```

Resources

https://github.com/ankitaggarwal011/InstrumentationOscilloscope?tab=readme-ov-file
https://github.com/ankitaggarwal011/InstrumentationOscilloscope/blob/master/Freescale%20FRDM-KL25Z/code/IOSv1_full/NOKIA_5110.lib
https://os.mbed.com/users/Rhyme/code/oscilloscope/
Build Affordable and Compact DIY ESP32 Oscilloscope
https://www.youtube.com/watch?v=BO8ipdXmHa4